

utils\filters.py

```
from typing import List, Dict, Any, Optional
from datetime import datetime, timezone, timedelta

class FilterService:
    """Service for filtering and pagination operations"""

    def filter_by_severity(self, cves: List[Dict], severity_filter: str) -> List[Dict]:
        """Filter CVEs by severity"""
        # Return original list if no filter specified
        if not severity_filter:
            return cves

        # Convert to uppercase for case-insensitive comparison
        severity_upper = severity_filter.upper()

        # Handle special "UNKNOWN" case - CVEs without standard severity ratings
        if severity_upper == 'UNKNOWN':
            return [
                cve for cve in cves
            ]
        # Check if severity is not one of the standard levels
        if severity_upper not in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
            return [
                cve for cve in cves
            ]
        # Handle standard severity levels
        elif severity_upper in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
            return [
                cve for cve in cves
                # Match the exact severity level (case-insensitive)
                if cve.get('Severity', '').upper() == severity_upper
            ]
        # Return unfiltered list if filter doesn't match any known severity
        return cves

    def filter_by_search(self, cves: List[Dict], search_query: str) -> List[Dict]:
        """Filter CVEs by search query"""
        # Return original list if no search query provided
        if not search_query:
            return cves

        # Convert to lowercase and remove whitespace for consistent matching
        q_lower = search_query.lower().strip()

        # Special handling for CWE (Common Weakness Enumeration) searches
        # CWE format: "cwe-" followed by digits (e.g., "cwe-79")
        if q_lower.startswith("cwe-") and q_lower[4:].isdigit():
            return [
                cve for cve in cves
                # Exact match for CWE field
                if (cve.get("CWE") or "").lower() == q_lower
            ]

        # General text search in ID and Description
        filtered_cves = []
        for cve in cves:
            # Get fields and convert to lowercase for case-insensitive search
            cve_id = cve.get('ID', '').lower()
            description = cve.get('Description', '').lower()
            cwe = cve.get('CWE', '').lower()

            # Check if search query appears in any of the searchable fields
            if (q_lower in cve_id or
                q_lower in description or
                q_lower in cwe):
                filtered_cves.append(cve)

        return filtered_cves

    def generate_page_numbers(self, current_page: int, total_pages: int) -> List[int]:
        """Generate page numbers for pagination"""
        # If 7 or fewer pages, show all page numbers
        if total_pages <= 7:
            return list(range(1, total_pages + 1))
```

```

# If user is on early pages (1-4), show first 7 pages
if current_page <= 4:
    return list(range(1, 8))
# If user is on final pages, show last 7 pages
elif current_page >= total_pages - 3:
    return list(range(total_pages - 6, total_pages + 1))
# For middle pages, show 7-page window centered on current page
else:
    return list(range(current_page - 3, current_page + 4))

def generate_note_text(self, year: Optional[int] = None,
month: Optional[int] = None,
day: Optional[int] = None) -> str:
    """Generate note text for data range"""
    # Most specific: year, month, and day provided
    if year and month and day:
        return f"Showing data from {year}-{month:02d}-{day:02d}"
    # Medium specific: year and month provided
    elif year and month:
        return f"Showing data from {year}-{month:02d}"
    # Least specific: only year provided
    elif year:
        return f"Showing data from {year}"
    # Default case: show 30-day rolling window
    else:
        # Use UTC timezone for consistency across deployments
        end_date = datetime.now(timezone.utc).date()
        # Calculate start date as 29 days ago (30 days total including today)
        start_date = end_date - timedelta(days=29)
        # Format dates in ISO format (YYYY-MM-DD)
        return f"Showing data from {start_date.strftime('%Y-%m-%d')} to {end_date.strftime('%Y-%m-%d')}"

```